

RUNTIME FOR EXECUTION & EXCHANGE ORCHESTRATION

# REXO

## Constituição Arquitetural

Especificação completa do núcleo, capabilities, execução, Economy Engine, memória, packs, marketplace, governança, Developer Experience, Studio visual e Platform Factory.



### Ylanna Almeida

Idealizadora e fundadora do projeto — visão original e direção de produto

|                 |                                        |
|-----------------|----------------------------------------|
| VERSÃO          | 3.0 · edição PT-BR                     |
| DATA            | Julho de 2026                          |
| STATUS          | Core v1 congelado · Fase 0 em execução |
| REDAÇÃO TÉCNICA | Colaboração com agentes de IA          |

GOVERNANÇA

# Controle do documento

| Campo                 | Definição                                                                              |
|-----------------------|----------------------------------------------------------------------------------------|
| Título                | REXO - Constituição Arquitetural                                                       |
| Versão                | 3.0 - edição em português do Brasil                                                    |
| Idealização           | Ylanna Almeida - visão original, direção de produto e fundação do projeto              |
| Colaboração           | Arquitetura e redação técnica desenvolvidas com OpenAI Codex                           |
| Estado                | Core v1 congelado; Fundação Pública v0.0.1 em execução incremental                     |
| Objetivo              | Definir os invariantes, componentes, contratos e modelo de evolução da plataforma      |
| Horizonte             | Arquitetura preparada para evolução de longo prazo e centenas de especialistas lógicos |
| Primeira distribuição | REXO Education                                                                         |
| Fonte de verdade      | Este documento, ADRs aprovados, contratos versionados e políticas publicadas           |

Regra de interpretação. A visão original é de Ylanna Almeida. A conversa de descoberta permanece como contexto histórico. Este documento consolida e substitui decisões exploratórias quando houver conflito.

# Sumário

Controle do documento . . . . .

2

1. Mandato e visão . . . . .

8

1.1 Resultado pretendido . . . . .

8

1.2 Não objetivos . . . . .

8

2. Princípios arquiteturais . . . . .

8

2.1 A metáfora correta . . . . .

9

3. Arquitetura lógica . . . . .

11

3.1 Control Plane . . . . .

11

3.2 Execution Plane . . . . .

11

3.3 Knowledge Plane . . . . .

11

3.4 Quality and Governance Plane . . . . .

12

4. Modelo de capabilities . . . . .

12

4.1 Capability Specification . . . . .

12

4.2 Capability Implementation . . . . .

12

4.3 Capability Resolver . . . . .

12

4.4 Taxonomia . . . . .

13

5. Agentes, Skills e workflows . . . . .

13

5.1 Agentes . . . . .

13

5.2 Skills . . . . .

13

5.3 Workflows . . . . .

13

5.4 Regra de escolha . . . . .

14

6. Fluxo de execução . . . . .

14

6.1 Envelope de tarefa . . . . .

14

6.2 Escala para centenas de agentes . . . . .

15

7. Memória e conhecimento . . . . .

15

7.1 Context Service . . . . .

16

7.2 Promoção de conhecimento . . . . .

16

7.3 Base de conhecimento . . . . .

16

8. Tool Gateway e MCP . . . . .

16

8.1 Responsabilidades do gateway . . . . .

16

8.2 Classes de efeito . . . . .

17

8.3 ComfyUI . . . . .

17

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>9. Packs, plugins e marketplace</b>                | <b>19</b> |
| 9.1 Conteúdo de um pacote                             | 19        |
| 9.2 Segurança do marketplace                          | 19        |
| 9.3 Package Manager                                   | 19        |
| <b>10. Apps e Distribution Builder</b>                | <b>19</b> |
| 10.1 Exemplos futuros                                 | 20        |
| <b>11. REXO Education</b>                             | <b>20</b> |
| 11.1 Composição                                       | 20        |
| 11.2 Pipeline completo                                | 20        |
| 11.3 Quality gates educacionais                       | 21        |
| <b>12. Qualidade e avaliação</b>                      | <b>23</b> |
| 12.1 Definition of Done                               | 23        |
| <b>13. Segurança, isolamento e governança</b>         | <b>23</b> |
| 13.1 Trust boundaries                                 | 23        |
| 13.2 Regras mandatórias                               | 24        |
| 13.3 Human-in-the-loop                                | 24        |
| <b>14. Versionamento e reprodutibilidade</b>          | <b>24</b> |
| 14.1 Run Manifest                                     | 24        |
| <b>15. Economy Engine, observabilidade e operação</b> | <b>24</b> |
| 15.1 Economy Engine                                   | 25        |
| 15.2 Sequência de decisão econômica                   | 25        |
| 15.3 Regras econômicas                                | 25        |
| 15.4 Telemetria                                       | 25        |
| 15.5 Budget Manager                                   | 26        |
| 15.6 SLOs iniciais                                    | 26        |
| <b>16. Evolução contínua</b>                          | <b>26</b> |
| 16.1 Melhorias que podem ser propostas                | 26        |
| 16.2 Componentes protegidos                           | 26        |
| 16.3 Promoção                                         | 27        |
| <b>17. Estrutura de diretórios</b>                    | <b>29</b> |
| 17.1 Catálogo gerado                                  | 30        |
| <b>18. Contratos fundamentais</b>                     | <b>30</b> |
| 18.1 Extensão sem alteração do núcleo                 | 31        |
| <b>19. Riscos e controles</b>                         | <b>31</b> |

20. Estratégia de implementação futura . . . . .

20.1 Critério para extrair microserviços . . . . .

21. Como trabalharemos . . . . .

21.1 Processo decisório . . . . .

21.2 Regras para o Chief AI Architect . . . . .

21.3 Regras para entregas futuras . . . . .

22. Decisões arquiteturais aprovadas nesta proposta . . . . .

23. Developer & Creation Plane . . . . .

23.1 Invariante de formato único . . . . .

23.2 Componentes . . . . .

23.3 Regra de extensibilidade . . . . .

24. REXO SDK . . . . .

24.1 APIs conceituais . . . . .

24.2 Estratégia de linguagens . . . . .

24.3 Conformance Test Kit . . . . .

25. REXO CLI, scaffolding e generators . . . . .

25.1 Interface alvo . . . . .

25.2 Jornada de criação de Pack . . . . .

25.3 Generators . . . . .

26. REXO Studio e REXO Canvas . . . . .

26.1 Workflow Canvas . . . . .

26.2 Domain Canvas . . . . .

26.3 Round-trip fidelity . . . . .

27. Runtime Marketplace e lifecycle de instalação . . . . .

27.1 Instalação transacional . . . . .

27.2 Atualização e remoção . . . . .

27.3 Runtime Profiles . . . . .

28. REXO Creator . . . . .

28.1 Platform Intent Specification . . . . .

28.2 Outputs . . . . .

28.3 Limites . . . . .

28.4 Exemplo . . . . .

29. Modos operacionais e ativação . . . . .

29.1 Como ativar comigo agora . . . . .

29.2 Confirmação de ativação . . . . .

29.3 Interface futura da CLI . . . . . 42

29.4 Máquina de estados . . . . . 42

29.5 Como eu funcionarei no projeto . . . . . 42

**Apêndice A. Glossário . . . . . 44**

**Apêndice B. Checklist de aprovação . . . . . 44**

**Próximo passo . . . . . 46**

---

PARTE

# Fundação

Mandato, visão, não objetivos e os princípios que governam todas as decisões seguintes.

01



FUNDAÇÃO

# 1. Mandato e visão

REXO é uma plataforma operacional extensível para compor, executar, governar e distribuir capacidades de inteligência artificial. Seu núcleo fornece orquestração durável, resolução de capacidades, acesso seguro a ferramentas, memória governada, avaliação, observabilidade e gerenciamento de pacotes.

A plataforma não será definida por um conjunto fixo de agentes. Agentes, workflows, aplicações e distribuições serão construídos sobre contratos de capabilities. O objetivo é permitir que novos domínios sejam instalados sem alterar o núcleo.

Capability-first. Pack-native. Agent-optional.

## 1.1 Resultado pretendido

- Receber objetivos de alto nível e transformá-los em planos verificáveis e entregas finais.
- Compor especialistas, Skills, workflows, ferramentas, modelos e políticas dinamicamente.
- Permitir distribuições especializadas, como Education, Software Factory, Creative e Marketing.
- Aprender de forma governada e reutilizar conhecimento validado entre projetos.
- Suportar execução local, privada, híbrida e em nuvem.
- Escalar de uma instalação individual para uma plataforma multi-tenant.

## 1.2 Não objetivos

- Não construir um chatbot monolítico com um prompt gigantesco.
- Não mapear literalmente cada cargo empresarial para um processo permanente.
- Não permitir comunicação irrestrita entre agentes.
- Não acoplar workflows a fornecedores, modelos ou MCPs específicos.
- Não permitir autoalteração direta do núcleo, das políticas ou da segurança.
- Não iniciar com microserviços ou infraestrutura distribuída sem necessidade comprovada.

CONSTITUIÇÃO

# 2. Princípios arquiteturais

| Princípio                        | Consequência prática                                                              |
|----------------------------------|-----------------------------------------------------------------------------------|
| Capability-first                 | A unidade fundamental é uma capacidade contratual; agentes são composições.       |
| Contracts before implementations | Schemas e protocolos são definidos antes dos componentes concretos.               |
| Durable by default               | Execuções longas sobrevivem a reinícios, falhas, pausas e retomadas.              |
| Least privilege                  | Permissões pertencem à tarefa e expiram; não pertencem permanentemente ao agente. |
| Evidence over confidence         | Decisões críticas exigem fontes, testes, métricas ou artefatos verificáveis.      |
| Memory is governed data          | Memória possui proveniência, confiança, escopo, validade e retenção.              |



| Princípio                   | Consequência prática                                                                |
|-----------------------------|-------------------------------------------------------------------------------------|
| Quality is executable       | Critérios de qualidade são avaliadores e testes, não adjetivos em prompts.          |
| Provider independence       | O núcleo conhece capabilities, não fornecedores específicos.                        |
| Reproducibility             | Toda run registra versões, configurações, políticas, modelos, ferramentas e hashes. |
| Evolution through promotion | Melhorias passam por sandbox, evals, canário, observação e promoção.                |

## 2.1 A metáfora correta

O termo Operating System descreve o papel do REXO como runtime e plataforma de aplicações de IA. Ele não será um kernel de sistema operacional tradicional. A analogia orienta responsabilidades, mas não deve produzir complexidade artificial.

| Sistema operacional | REXO                                                                   |
|---------------------|------------------------------------------------------------------------|
| Processo            | Run ou task executada por worker efêmero                               |
| Scheduler           | Agendador de tarefas, prioridades, budgets e concorrência              |
| Driver              | Capability provider ou adapter do Tool Gateway                         |
| Aplicativo          | AI App ou Application Pack                                             |
| Pacote              | Provider, Capability, Domain, Policy, Application ou Distribution Pack |
| Sistema de arquivos | Artifact Store e Knowledge Plane                                       |
| Permissão           | Capability token e política de execução                                |
| Distribuição        | Composição de packs, apps, providers e políticas                       |

---

PARTE

# Arquitetura

Planos lógicos, modelo de capabilities, composição, runtime, memória e integrações.

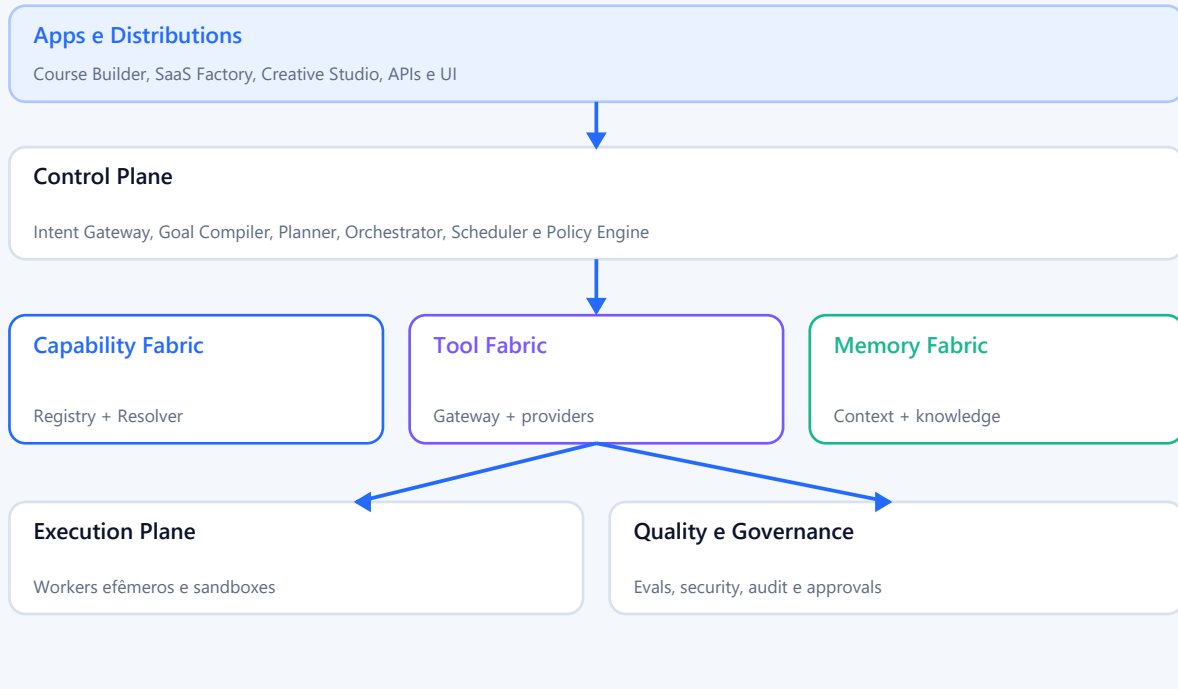
02



## VISÃO DO SISTEMA

## 3. Arquitetura lógica

### REXO — arquitetura por planos



### 3.1 Control Plane

- Intent Gateway: normaliza pedidos vindos de UI, API, CLI e automações.
- Goal Compiler: transforma linguagem natural em Goal Specification tipada.
- Planner: cria um DAG de tarefas, dependências, gates e critérios de aceitação.
- Plan Validator: verifica ciclos, permissões, riscos, custos e completude.
- Durable Orchestrator: controla estado, retries, timeouts, cancelamento e retomada.
- Scheduler: distribui tarefas conforme prioridade, budget, capacidade e recursos.
- Policy Engine: aplica segurança, aprovação, retenção, compliance e limites.

### 3.2 Execution Plane

Workers são efêmeros, stateless sempre que possível e executados em sandboxes proporcionais ao risco. Cada worker recebe contexto mínimo, Skills fixadas por versão, capability tokens temporários, budget e contrato de saída.

### 3.3 Knowledge Plane

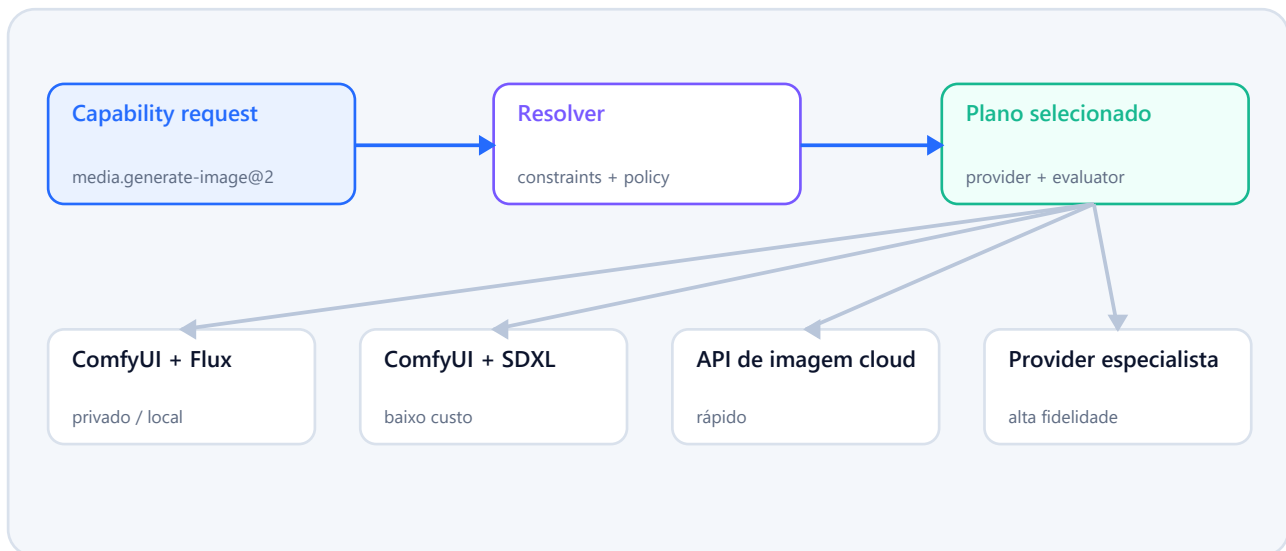
O Knowledge Plane separa registro oficial, artefatos, índices de busca e conhecimento curado. Índices vetoriais são derivados e reconstruíveis; nunca são a fonte oficial.

### 3.4 Quality and Governance Plane

A qualidade é transversal: valida o plano antes da execução, verifica outputs intermediários, executa regressões e controla a promoção de componentes.

#### NÚCLEO CONCEITUAL

## 4. Modelo de capabilities



### 4.1 Capability Specification

Contrato abstrato, versionado e independente de fornecedor. Define semântica, schemas de entrada e saída, efeitos possíveis, riscos, classes de qualidade, observabilidade mínima e requisitos de idempotência.

```
kind: CapabilitySpecification
apiVersion: rexo.dev/v1
name: media.generate-image
version: 2.0.0
inputSchema: schemas/generate-image.input.json
outputSchema: schemas/image-artifact.output.json
effects: [artifact.write]
riskClass: medium
qualityProfile: visual-image-v2
```

### 4.2 Capability Implementation

Uma implementação declara como satisfaz a especificação: provider, modelos, tools, Skills, custos, latência, regiões, restrições e avaliadores.

### 4.3 Capability Resolver

- Filtra implementações incompatíveis com políticas, permissões, região ou hardware.
- Elimina opções indisponíveis ou fora do budget.
- Ordena candidatos por qualidade prevista, custo, latência, privacidade e histórico.
- Pode executar competição entre provedores quando o valor do resultado justificar.
- Seleciona avaliadores compatíveis e produz um Resolution Record auditável.

- Aplica fallback apenas quando o contrato permite e registra a substituição.

### 4.4 Taxonomia

Capabilities usarão namespaces estáveis, como `education.*`, `media.*`, `software.*`, `knowledge.*`, `quality.*` e `publishing.*`. A taxonomia descreve resultado, não tecnologia.

#### COMPOSIÇÃO

## 5. Agentes, Skills e workflows

### 5.1 Agentes

Agent Definition é um papel lógico composto por políticas, capacidades decisórias, Skill references, perfil de memória, budgets e critérios de delegação. Instâncias de execução não mantêm personalidade ou memória ilimitada.

| Agente contém                   | Agente não contém                |
|---------------------------------|----------------------------------|
| Objetivo e limites de decisão   | Credenciais permanentes          |
| Capabilities que pode solicitar | Implementações fixas de provider |
| Política de memória             | Workflow global embutido         |
| Regras de delegação             | Conhecimento duplicado de Skills |
| Quality profile                 | Estado operacional permanente    |

### 5.2 Skills

Skill é uma unidade versionada de competência. Deve possuir manifesto, instruções, schemas, recursos, scripts opcionais, exemplos, testes, evals e changelog.

```
skill/  
|-- skill.yaml  
|-- instructions.md  
|-- input.schema.json  
|-- output.schema.json  
|-- resources/  
|-- scripts/  
|-- templates/  
|-- examples/  
|-- tests/  
|-- evals/  
`-- CHANGELOG.md
```

### 5.3 Workflows

Workflow é um DAG durável e versionado. Ele coordena capabilities, tarefas determinísticas, aprovações, compensações e avaliações. Skills não controlam o workflow global; agentes não substituem o orchestrator.

## 5.4 Regra de escolha

| Necessidade                                   | Componente adequado |
|-----------------------------------------------|---------------------|
| Competência reutilizável                      | Skill               |
| Resultado abstrato independente de fornecedor | Capability          |
| Sequência durável e auditável                 | Workflow            |
| Decisão contextual dentro de limites          | Agent Definition    |
| Operação externa                              | Tool Provider       |
| Experiência completa para usuário             | App                 |

RUNTIME

## 6. Fluxo de execução



1. **Compilar:** Converter objetivo em Goal Specification com entregáveis, restrições, budget, risco e critérios de aceitação.
2. **Planejar:** Produzir DAG tipado com dependências, tarefas, gates, retries, timeouts e artefatos esperados.
3. **Validar:** Rejeitar planos incompletos, inseguros, cíclicos, excessivos ou sem critérios verificáveis.
4. **Resolver:** Selecionar capabilities, implementações, Skills, workers, providers e avaliadores.
5. **Executar:** Distribuir tasks para workers efêmeros com leases, tokens temporários e contexto mínimo.
6. **Avaliar:** Executar testes determinísticos, rubricas, revisão independente e aprovações proporcionais ao risco.
7. **Corrigir:** Criar tarefas específicas para falhas; não reiniciar todo o pipeline sem necessidade.
8. **Entregar:** Montar pacote final apenas com artefatos aprovados e relatório de proveniência.
9. **Aprender:** Gerar candidatos a aprendizado; nenhuma promoção ocorre diretamente durante a run.

### 6.1 Envelope de tarefa

Toda comunicação interagente ocorrerá por Task Envelope ou Artifact Contract. Conversas livres não são o mecanismo operacional oficial.

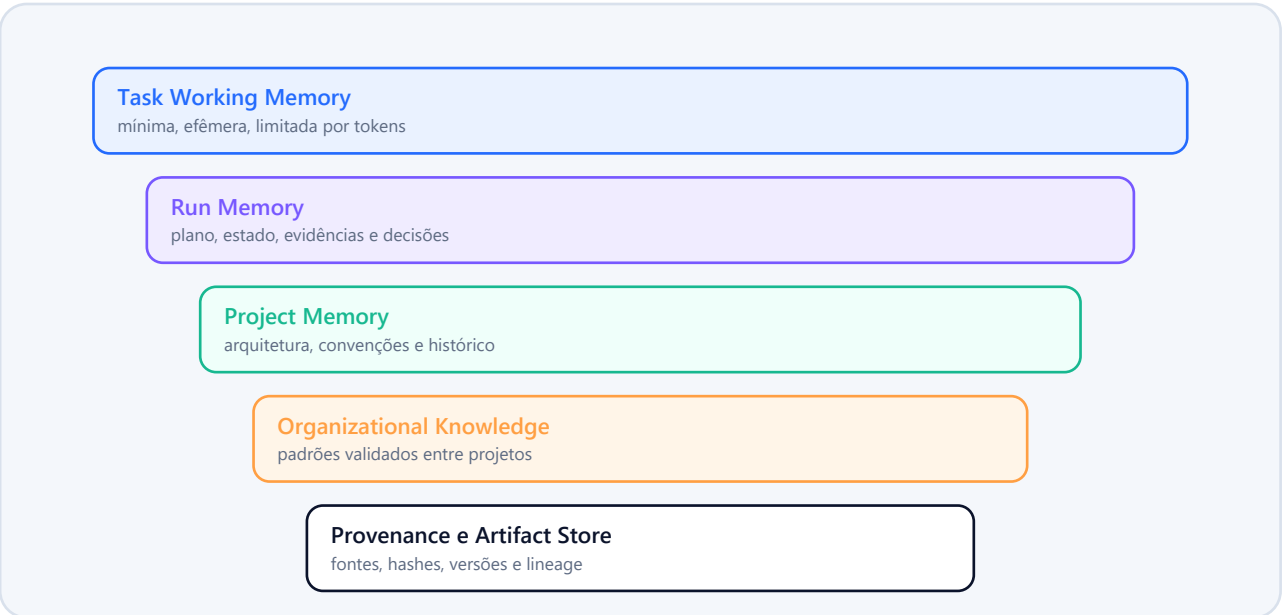
```
taskId
runId
parentTaskId
capability
inputArtifacts
expectedOutputSchema
acceptanceCriteria
skillLock
memoryPolicy
toolPolicy
budget
timeout
retryPolicy
riskClass
traceContext
```

### 6.2 Escala para centenas de agentes

Centenas de agentes significam centenas de definições disponíveis no catálogo. O Scheduler cria apenas as instâncias necessárias. Backpressure, quotas, leases, filas de prioridade e limites de fan-out impedem explosão de execução.

MEMORY FABRIC

## 7. Memória e conhecimento



| Camada           | Conteúdo                                | Retenção               | Acesso                      |
|------------------|-----------------------------------------|------------------------|-----------------------------|
| Task working     | Contexto mínimo de uma tarefa           | Efêmera                | Worker atual                |
| Run              | Plano, estado, decisões e evidências    | Vida da run + retenção | Componentes da run          |
| Project          | Arquitetura, padrões e histórico        | Longa                  | Política do projeto         |
| Agent-type       | Heurísticas validadas por tipo e versão | Versionada             | Agent Definition compatível |
| User preferences | Idioma, estilo, limites e preferências  | Longa e editável       | Policy-controlled           |

| Camada              | Conteúdo                             | Retenção          | Acesso                  |
|---------------------|--------------------------------------|-------------------|-------------------------|
| Organization        | Conhecimento validado entre projetos | Longa e curada    | Escopo organizacional   |
| Artifact/provenance | Arquivos, hashes, fontes e lineage   | Conforme política | Referências autorizadas |

### 7.1 Context Service

- Recebe tarefa, escopo e budget de tokens.
- Consulta somente memórias autorizadas.
- Prioriza fatos com fonte, validade e confiança.
- Remove duplicações e conteúdo obsoleto.
- Produz um Context Manifest que registra o que foi fornecido ao worker.
- Impede que dados de outro tenant ou projeto sejam incluídos.

### 7.2 Promoção de conhecimento

Observações viram candidatos. Candidatos são deduplicados, classificados, verificados, avaliados e promovidos. Conhecimento organizacional exige proveniência e não pode ser escrito diretamente por um worker.

### 7.3 Base de conhecimento

O pipeline de ingestão preserva documento bruto, versão normalizada, chunks derivados, claims extraídos, metadados, embeddings e relações. Toda resposta factual deve ser capaz de retornar à fonte original.

#### INTEGRAÇÕES

## 8. Tool Gateway e MCP

MCP é um protocolo de acesso a ferramentas, não a arquitetura da plataforma. REXO abstrai MCPs, APIs, CLIs, aplicações locais e serviços cloud como providers de capabilities.

### 8.1 Responsabilidades do gateway

- Descoberta, health checks e registro de providers.
- Isolamento e rotação de credenciais.
- Capability tokens temporários e least privilege.
- Rate limits, timeouts, circuit breakers e retries seguros.
- Normalização de inputs, outputs e erros.
- Auditoria de todos os efeitos externos.
- Aprovação antes de ações destrutivas, financeiras, públicas ou irreversíveis.
- Fallback controlado e registrado.



## 8.2 Classes de efeito

| Classe          | Exemplo                             | Política típica             |
|-----------------|-------------------------------------|-----------------------------|
| Read            | Consultar documentação ou banco     | Permitido pelo escopo       |
| Workspace write | Criar ou editar arquivo             | Permitido dentro do projeto |
| External write  | Enviar mensagem ou atualizar SaaS   | Autorização específica      |
| Publish         | Publicar curso, site ou vídeo       | Gate de aprovação           |
| Financial       | Compra ou consumo excepcional       | Aprovação obrigatória       |
| Destructive     | Excluir dados ou substituir recurso | Aprovação e backup          |
| Administrative  | Permissões, usuários e credenciais  | Controle privilegiado       |

## 8.3 ComfyUI

ComfyUI será um provider multimídia. Seleção de modelo, workflow, LoRA, VAE, sampler, scheduler, ControlNet, upscaler e otimização de VRAM serão capabilities separadas. O sistema poderá gerar alternativas, avaliar resultados e regenerar somente o necessário.



PARTE

# Ecosystem

Packs, distribuições, apps e a primeira vertical de referência: REXO Education.

03



ECOSSISTEMA

## 9. Packs, plugins e marketplace

| Tipo              | Responsabilidade                          | Exemplos                              |
|-------------------|-------------------------------------------|---------------------------------------|
| Provider Pack     | Integrações e implementações concretas    | ComfyUI, OpenAI, Blender, YouTube     |
| Capability Pack   | Competências reutilizáveis                | Video Production, Research, Documents |
| Domain Pack       | Conhecimento, policies e evals de domínio | Education, Legal, Finance             |
| Policy Pack       | Compliance e governança                   | LGPD, Enterprise Security             |
| Application Pack  | Experiência completa para usuário         | Course Builder, SaaS Factory          |
| Distribution Pack | Composição publicável de packs e apps     | REXO Education                        |

### 9.1 Conteúdo de um pacote

- Manifesto, publisher, assinatura, licença e versão.
- Capabilities fornecidas e dependências exigidas.
- Skills, workflows, templates, avaliadores e Agent Definitions.
- Providers e MCP adapters opcionais.
- Permissions manifest e políticas de dados.
- Compatibility ranges, migrations e uninstall plan.
- SBOM, changelog, eval baselines e trust tier.

### 9.2 Segurança do marketplace

Pacotes de terceiros são software não confiável. Instalação deve resolver dependências sem executar hooks arbitrários no Control Plane. Packs serão assinados, verificados, classificados por confiança e executados em sandbox compatível com suas permissões.

### 9.3 Package Manager

- Instala, atualiza, desativa, remove e reverte pacotes.
- Mantém lockfile por distribuição.
- Detecta conflitos de capabilities, schemas, policies e versões.
- Executa migrations transacionais e suporta rollback.
- Gera catálogo resolvido, compatibility report e permission diff.

PLATAFORMA PARA PLATAFORMAS

## 10. Apps e Distribution Builder

Apps são experiências orientadas ao usuário. Distribuições combinam Apps, Packs, Providers, Policies, configurações e identidade. O Distribution Builder permite criar plataformas verticais sem modificar o REXO Core.

```
distribution:
  name: REXO Education
  coreCompatibility: ">=1.0 <2.0"
  packs:
    - education-domain
    - course-production
    - creative-media
    - publishing
    - learning-analytics
  providers:
    - comfyui
    - document-runtime
    - voice-provider
  policies:
    - content-provenance
    - commercial-licensing
    - child-safety
  apps:
    - course-builder
    - education-admin
```

10.1 Exemplos futuros

- REXO Software Factory: arquitetura, implementação, QA, DevOps e observabilidade.
- REXO Creative: imagem, vídeo, voz, animação, design e publicação.
- REXO Marketing: pesquisa, estratégia, campanhas, conteúdo, mídia e analytics.
- REXO GameDev: design, narrativa, arte, código, áudio, testes e distribuição.
- REXO Enterprise: integrações internas, compliance, dados privados e automações.

DISTRIBUIÇÃO DE REFERÊNCIA

11. REXO Education

REXO Education será o primeiro vertical e o laboratório de validação da arquitetura. Ele exercita pesquisa, planejamento, documentos, código, mídia, QA, publicação e analytics sem introduzir regras educacionais no núcleo.

11.1 Composição

| Componente             | Escopo                                                                   |
|------------------------|--------------------------------------------------------------------------|
| Education Domain Pack  | Pedagogia, currículo, rubricas, taxonomia, policies e learning analytics |
| Course Production Pack | Pesquisa, objetivos, módulos, aulas, roteiros, avaliações e packaging    |
| Creative Media Packs   | Slides, imagens, diagramas, voz, animação, vídeo e thumbnails            |
| Publishing Pack        | SEO, landing pages, LMS, vídeo, arquivos e metadados                     |
| Course Builder App     | Criação, acompanhamento, previews, aprovações e publicação               |
| Education Admin App    | Catálogo, custos, desempenho, qualidade, alunos e evolução               |

11.2 Pipeline completo

1. Discovery e definição do público.

2. Pesquisa com fontes e validação técnica.
3. Learning objectives e desenho curricular.
4. Módulos, aulas, dependências e estimativa de carga.
5. Roteiros, exemplos, demonstrações e storyboards.
6. Slides, diagramas, PDFs, imagens, voz, animação e vídeo.
7. Exercícios, quizzes, projetos, gabaritos e rubricas.
8. QA técnico, pedagógico, audiovisual, legal e de acessibilidade.
9. Landing page, SEO, thumbnails, Shorts, Reels e materiais de divulgação.
10. Publicação e empacotamento para destinos configurados.
11. Analytics de produção e aprendizagem.
12. Propostas de melhoria curricular baseadas em evidência.

### 11.3 Quality gates educacionais

- Cobertura dos objetivos de aprendizagem.
- Correção e atualidade técnica.
- Progressão de dificuldade e pré-requisitos.
- Clareza, exemplos e carga cognitiva.
- Alinhamento entre aula, exercício e avaliação.
- Legibilidade, áudio, sincronização, acessibilidade e direitos de uso.
- Proveniência das fontes e dos ativos.

---

PARTE

# Qualidade, Economia e Governança

Quality Engine, segurança, reprodutibilidade, Economy Engine e evolução controlada.

04



QUALITY ENGINE

## 12. Qualidade e avaliação

Não haverá uma nota universal como 9,5. Cada artifact type e risk class possui um Quality Profile com validadores obrigatórios, thresholds, avaliadores, amostragem e regras de aprovação.

| Camada              | Uso                                                                      |
|---------------------|--------------------------------------------------------------------------|
| Determinístico      | Schema, compilação, lint, testes, hashes, duração, formato               |
| Model-based         | Rubricas de clareza, coerência, estética e completude                    |
| Comparativo         | Selecionar melhor opção entre candidatos                                 |
| Independent review  | Revisor sem acesso à justificativa do produtor quando apropriado         |
| Human gate          | Riscos altos, publicação, custo, direitos, medicina, jurídico e finanças |
| Regression          | Comparar nova versão com baseline e golden datasets                      |
| Production feedback | Erros reais, satisfação, performance e learning analytics                |

### 12.1 Definition of Done

- Output satisfaz schema e critérios de aceitação.
- Testes obrigatórios foram aprovados.
- Fontes e proveniência estão presentes.
- Não existem violações de policy.
- Custos e limitações foram registrados.
- Artefatos foram versionados e possuem hashes.
- Aprovações exigidas foram obtidas.
- Pacote final contém relatório de qualidade.

TRUST

## 13. Segurança, isolamento e governança

### 13.1 Trust boundaries

- Usuário e tenant.
- Projeto.
- Run.
- Worker e sandbox.
- Pack e publisher.
- Provider e ferramenta externa.
- Memory scope.
- Artifact visibility.

### 13.2 Regras mandatórias

| Área           | Regra                                                                           |
|----------------|---------------------------------------------------------------------------------|
| Credenciais    | Nunca entram em prompts, logs, artifacts ou memórias; são injetadas no gateway. |
| Permissões     | Concedidas por capability token, tarefa, escopo e tempo.                        |
| Dados          | Classificados, criptografados, retidos e excluídos por policy.                  |
| Plugins        | Assinados, analisados, permissionados e isolados.                               |
| Auditoria      | Eventos relevantes são imutáveis e correlacionados por trace.                   |
| Ações críticas | Exigem gates explícitos e registro do aprovador.                                |
| Modelos        | Recebem somente o contexto mínimo necessário.                                   |
| Supply chain   | Dependências e artefatos possuem SBOM e hashes.                                 |

### 13.3 Human-in-the-loop

Autonomia é configurável por domínio, risco e tenant. O sistema deve distinguir perguntas de esclarecimento, aprovações de plano, autorizações de efeito externo e revisões especializadas.

#### LIFECYCLE

## 14. Versionamento e reprodutibilidade

- SemVer: Agent Definitions, Skills, capabilities, workflows, packs, providers, policies e evaluators.
- Schema versioning: compatibilidade explícita e migrations.
- Lockfiles: resolução completa de uma distribuição e de uma run.
- Immutable artifacts: outputs publicados não são sobrescritos.
- Content hashes: integridade e deduplicação.
- Release channels: development, canary e stable.
- ADRs: decisões arquiteturais e suas consequências.
- RFCs: mudanças amplas, contratos e extensões do ecossistema.

### 14.1 Run Manifest

Toda execução registra versões do Core, distribuição, packs, Skills, Agent Definitions, workflows, policies, modelos, prompts, providers, tools, dados de entrada, context manifest, outputs, avaliadores, custos e eventos. Isso permite auditoria e reprodução proporcional à natureza não determinística dos modelos.

#### OPERAÇÕES

## 15. Economy Engine, observabilidade e operação



## 15.1 Economy Engine

### Economy Engine — sequência de decisão



O Economy Engine é o componente que decide, antes de qualquer chamada paga ou probabilística, se ela deve de fato acontecer. Ele atravessa o Control Plane e o Scheduler e se aplica a toda operação que consome tokens, chamadas de modelo, providers pagos ou tempo de computação relevante.

**Reutilizar antes de gerar. Gerar o mínimo necessário. Nunca abaixo do piso de qualidade.**

## 15.2 Sequência de decisão econômica

1. Existe um artefato válido e com fingerprint compatível já produzido? Reutilizar em vez de recriar.
2. É possível uma reconstrução incremental, recalculando apenas a parte que mudou?
3. Qual é o contexto mínimo necessário para a tarefa, sem enviar o repositório ou o histórico inteiro?
4. Qual é o menor provider ou modelo capaz de atingir o quality profile exigido?
5. Quanto resta de orçamento em tokens, chamadas, tempo e custo para esta run, workflow e task?
6. Qual é a qualidade esperada do resultado e qual é o fallback caso ela não seja atingida?

## 15.3 Regras econômicas

- Otimização de custo nunca pode cruzar o piso mínimo de qualidade declarado para a capability.
- Workers efêmeros recebem apenas o contexto necessário para a tarefa; contexto amplo é uma exceção justificada, não o padrão.
- Cache keys incluem todas as entradas relevantes de política, para impedir reuso inseguro de artifacts.
- Toda decisão de reuso, incremental rebuild ou downgrade de provider é registrada no Resolution Record da run.
- Competição entre providers só ocorre quando o valor esperado do resultado justificar o custo adicional.

## 15.4 Telemetria

- Traces por objetivo, run, task, capability, provider e evaluator.
- Logs estruturados sem segredos ou dados indevidos.
- Métricas de qualidade, custo, latência, retries, falhas e cache.
- Lineage entre inputs, decisões, artifacts e publicações.
- Dashboards operacionais e alertas por SLO.

## 15.5 Budget Manager

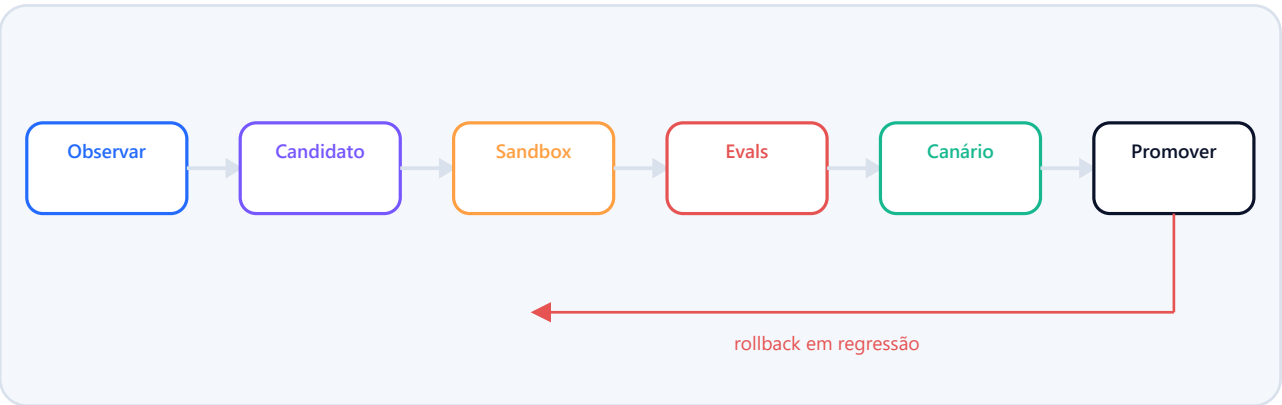
Budgets serão hierárquicos: tenant, projeto, run, workflow e task. O scheduler reserva, consome e devolve orçamento. Desvios, competições entre providers e regenerações precisam de limites explícitos.

## 15.6 SLOs iniciais

| Dimensão    | Indicadores                                          |
|-------------|------------------------------------------------------|
| Reliability | Conclusão, recuperação, idempotência e taxa de falha |
| Quality     | Aprovação, regressão, correção e satisfação          |
| Performance | Fila, execução, provider e tempo total               |
| Cost        | Custo por task, artifact, workflow e entrega         |
| Security    | Violações, acessos negados, packages e secrets       |
| Knowledge   | Freshness, provenance, reutilização e contaminação   |

### LEARNING ENGINE

# 16. Evolução contínua



Autoevolução significa produzir e testar candidatos a melhoria. Não significa permitir que o sistema altere sua própria fundação diretamente.

## 16.1 Melhorias que podem ser propostas

- Skills, templates, prompts, rubricas e exemplos.
- Heurísticas de resolução e model routing.
- Novas composições de agentes.
- Otimizações de workflows.
- Avaliadores e golden datasets.
- Novos providers e capability implementations.
- Documentação e conhecimento curado.

## 16.2 Componentes protegidos

- Políticas centrais de segurança e autorização.
- Mecanismo de auditoria.

- Pipeline de promoção.
- Credenciais e trust roots.
- Contratos fundamentais do Core.
- Gates humanos mandatórios.

## 16.3 Promoção

Candidatos são construídos em sandbox, comparados com baseline, submetidos a regressões, segurança e custo, liberados em canário, observados e então promovidos ou revertidos.

---

PARTE

# Estrutura e Execução

Organização do repositório, contratos, riscos, roadmap e modelo de trabalho.

05



## MONOREPO MODULAR

## 17. Estrutura de diretórios

O repositório contém contratos e componentes versionados. Dados operacionais, memórias, artifacts, secrets e caches permanecem fora do Git.

```
rexo/  
|-- README.md  
|-- ARCHITECTURE.md  
|-- SECURITY.md  
|-- GOVERNANCE.md  
|-- CONTRIBUTING.md  
|-- VERSION  
|-- docs/  
|   |-- vision/  
|   |-- architecture/  
|   |-- adr/  
|   |-- rfc/  
|   |-- governance/  
|   |-- runbooks/  
|   `-- threat-models/  
|-- contracts/  
|   |-- capability/  
|   |-- agent/  
|   |-- skill/  
|   |-- workflow/  
|   |-- task/  
|   |-- event/  
|   |-- artifact/  
|   |-- memory/  
|   |-- evaluation/  
|   |-- tool/  
|   `-- package/  
|-- platform/  
|   |-- control-plane/  
|   |   |-- intent-gateway/  
|   |   |-- goal-compiler/  
|   |   |-- planner/  
|   |   |-- plan-validator/  
|   |   |-- orchestrator/  
|   |   |-- scheduler/  
|   |   |-- registry/  
|   |   `-- policy-engine/  
|   |-- execution-plane/  
|   |   |-- worker-runtime/  
|   |   |-- sandbox-manager/  
|   |   |-- context-service/  
|   |   `-- artifact-service/  
|   |-- knowledge-plane/  
|   |   |-- memory-service/  
|   |   |-- knowledge-service/  
|   |   |-- ingestion-service/  
|   |   |-- provenance-service/  
|   |   `-- retrieval-service/  
|   |-- quality-plane/  
|   |   |-- evaluation-engine/  
|   |   |-- review-engine/  
|   |   |-- regression-runner/  
|   |   `-- approval-service/  
|   |-- capability-fabric/  
|   |-- tool-gateway/  
|   |-- package-manager/  
|   |-- eventing/  
|   |-- observability/  
|   |-- security/  
|   `-- cost-management/  
|-- components/  
|   |-- agents/<domain>/<name>/  
|   |-- skills/<domain>/<name>/  
|   |-- workflows/<domain>/<name>/
```

```
| |-- evaluators/<domain>/<name>/
| |-- templates/<domain>/<name>/
| |-- providers/<provider>/
| `-- packages/<package>/
|-- distributions/
|   |-- education/
|-- policies/
|-- sdk/
|   |-- capability-sdk/
|   |-- agent-sdk/
|   |-- skill-sdk/
|   |-- workflow-sdk/
|   |-- package-sdk/
|   |-- evaluator-sdk/
|   `-- testing-sdk/
|-- catalog/
|   |-- taxonomy/
|   |-- compatibility/
|   `-- generated/
|-- evals/
|-- tests/
|-- apps/
|   |-- api/
|   |-- cli/
|   |-- admin-console/
|   |-- operator-console/
|   `-- marketplace/
|-- infrastructure/
`-- examples/
```

## 17.1 Catálogo gerado

Manifestos dos componentes são a fonte oficial. O diretório de catálogo gerado é um índice reconstruível e não deve duplicar manualmente definições.

### EXTENSIBILIDADE

# 18. Contratos fundamentais

| Contrato                  | Função                                                                    |
|---------------------------|---------------------------------------------------------------------------|
| Goal Specification        | Objetivo, entregáveis, constraints, budgets, riscos e acceptance criteria |
| Plan DAG                  | Tasks, dependências, gates, retries, timeouts e outputs                   |
| Task Envelope             | Contrato de execução entre orchestrator e worker                          |
| Capability Specification  | Resultado abstrato e seus schemas                                         |
| Capability Implementation | Provider concreto e suas restrições                                       |
| Artifact Manifest         | Tipo, hash, versão, lineage, fontes e status                              |
| Memory Record             | Escopo, validade, confiança, proveniência e retenção                      |
| Evaluation Report         | Critérios, evidências, scores, falhas e decisão                           |
| Package Manifest          | Conteúdo, dependências, permissões e compatibilidade                      |
| Run Manifest              | Resolução completa e histórico da execução                                |
| Event Envelope            | Evento versionado, idempotente e correlacionado                           |

### 18.1 Extensão sem alteração do núcleo

Novos agentes, Skills, workflows, providers, avaliadores, packs e Apps são descobertos por manifestos compatíveis. O Core não recebe condicionais específicas de domínio. Mudanças no Core só são justificadas quando um contrato geral é insuficiente.

RISK REGISTER

## 19. Riscos e controles

| Risco               | Impacto                           | Controle                                              |
|---------------------|-----------------------------------|-------------------------------------------------------|
| Explosão de tarefas | Fan-out e custos crescentes       | Budgets, profundidade, fusão de tasks e backpressure  |
| Loops sem progresso | Retries e revisões infinitas      | Limites, progress detector e escalonamento            |
| Memory poisoning    | Erro persistente entre projetos   | Proveniência, curadoria, escopo e promoção            |
| Skill duplication   | Catálogo confuso e inconsistente  | Taxonomia, overlap detection, ownership e depreciação |
| Supply chain        | Pacotes maliciosos ou vulneráveis | Assinatura, SBOM, sandbox, trust tiers e scanning     |
| Provider lock-in    | Workflows presos a fornecedor     | Capability contracts e adapters                       |
| Quality gaming      | Métrica boa e resultado ruim      | Evals variados, revisão independente e feedback real  |
| Knowledge staleness | Conteúdo desatualizado            | Freshness, TTL, revalidação e fontes autoritativas    |
| Cost opacity        | Paralelismo e regeneração         | Budget manager, quotas e estimativa                   |
| Overengineering     | Infraestrutura antes do produto   | Monólito modular e primeiro vertical                  |
| Data leakage        | Contexto entre tenants            | Isolation, least privilege e context manifest         |
| Model churn         | Mudança frequente de modelos      | Resolver, compatibility matrix e regression suite     |

ROADMAP

## 20. Estratégia de implementação futura

A ambição é ampla; a implementação inicial deve ser estreita. A regra será arquitetura universal com primeira distribuição vertical.

| Fase                        | Resultado                                                                                  |
|-----------------------------|--------------------------------------------------------------------------------------------|
| Fase 0 - Constituição       | Aprovar princípios, contratos, ADRs, taxonomia e threat model.                             |
| Fase 1 - Kernel mínimo      | Runs, tasks, durable orchestration, registry, worker runtime, artifacts e observabilidade. |
| Fase 2 - Capability Fabric  | Specifications, implementations, resolver, policies, Tool Gateway e lockfiles.             |
| Fase 3 - Memory and Quality | Context Service, Project Memory, provenance, evaluators e quality gates.                   |
| Fase 4 - Education MVP      | Research, curriculum, lessons, scripts, documents e QA; mídia inicialmente controlada.     |
| Fase 5 - Creative providers | ComfyUI, voz, vídeo, diagrams, thumbnails e comparative evaluation.                        |
| Fase 6 - Package Manager    | Packs instaláveis, dependencies, permissions, migrations e rollback.                       |
| Fase 7 - REXO Education     | Course Builder, publishing, analytics e operação end-to-end.                               |

| Fase                          | Resultado                                                                |
|-------------------------------|--------------------------------------------------------------------------|
| Fase 8 - Marketplace privado  | Publisher workflow, trust tiers, signing, review e distribuição interna. |
| Fase 9 - Platform factory     | Distribution Builder, SDKs e novas distribuições.                        |
| Fase 10 - Marketplace público | Somente após maturidade de sandbox, security e governance.               |

## 20.1 Critério para extrair microserviços

Componentes só serão separados fisicamente quando houver demanda comprovada de escala, isolamento, segurança, ownership ou ciclo de deploy. Modularidade lógica vem antes de distribuição física.

### OPERATING MODEL

# 21. Como trabalharemos

## 21.1 Processo decisório

1. Definir problema, contexto, invariantes e alternativas.
2. Registrar decisões relevantes em ADR.
3. Usar RFC para mudanças que afetem contratos, ecossistema ou segurança.
4. Construir o menor incremento que teste a hipótese arquitetural.
5. Criar evals e critérios antes ou junto da implementação.
6. Medir qualidade, custo, latência, segurança e operabilidade.
7. Promover somente após evidência; reverter quando houver regressão.

## 21.2 Regras para o Chief AI Architect

- Criticar ideias, inclusive as do usuário, quando houver risco ou solução superior.
- Evitar decisões irreversíveis prematuras.
- Preservar separação de responsabilidades e contratos.
- Não criar abstrações sem pelo menos um caso real e um segundo caso plausível.
- Não transformar preferência de provider em decisão de arquitetura.
- Exigir segurança, observabilidade, migração e rollback em recursos extensíveis.
- Tratar conhecimento e qualidade como produtos, não acessórios.
- Manter documentação e ADRs sincronizados com o sistema.

## 21.3 Regras para entregas futuras

- Nenhum componente é concluído sem testes e evals proporcionais ao risco.
- Nenhuma capability é publicada sem schemas e implementação de referência.
- Nenhum pack é instalado sem permission diff e compatibility check.
- Nenhuma ação externa crítica ocorre sem autorização correspondente.
- Nenhuma melhoria automática chega a stable sem regressão e canário.
- Nenhum conhecimento organizacional é promovido sem proveniência.



BASELINE

## 22. Decisões arquiteturais aprovadas nesta proposta

| ID     | Decisão                             | Motivo                                                                             |
|--------|-------------------------------------|------------------------------------------------------------------------------------|
| AD-001 | REXO, não AI Company                | A empresa de agentes é uma experiência possível; o produto é o runtime extensível. |
| AD-002 | Capability-first                    | Capabilities são contratos centrais; agentes são composições opcionais.            |
| AD-003 | Core estável e domínios instaláveis | Education e demais verticais não alteram o núcleo.                                 |
| AD-004 | Durable orchestration               | Workflows longos são persistentes, retomáveis e auditáveis.                        |
| AD-005 | Central Tool Gateway                | MCPs e tools não são acessados diretamente por agentes.                            |
| AD-006 | Layered memory                      | Sem memória global irrestrita; contexto é montado por política.                    |
| AD-007 | Executable quality                  | Evals e gates substituem notas genéricas.                                          |
| AD-008 | Controlled evolution                | Autoevolução passa por promoção e rollback.                                        |
| AD-009 | Modular monolith first              | Distribuição física somente mediante necessidade comprovada.                       |
| AD-010 | Education as reference distribution | Primeiro produto valida o Core sem contaminá-lo.                                   |
| AD-011 | Typed package ecosystem             | Packs possuem tipos, contratos, permissões e lifecycle.                            |
| AD-012 | Platform for platforms              | Distribution Builder compõe plataformas especializadas.                            |
| AD-013 | Developer & Creation Plane          | SDK, CLI, Studio, Canvas e Creator formam uma camada oficial.                      |
| AD-014 | One contract model                  | Todas as superfícies produzem os mesmos manifests e schemas.                       |
| AD-015 | Studio contains Canvas              | Uma interface visual unificada evita produtos e formatos paralelos.                |
| AD-016 | Runtime-aware marketplace           | Packs podem resolver ambientes de execução assinados e isolados.                   |
| AD-017 | Creator is not privileged           | A Platform Factory usa o SDK e não publica diretamente em stable.                  |
| AD-018 | Explicit operating modes            | Transições de arquitetura, engenharia e operação são declaradas e gated.           |

Estado final: esta versão define a arquitetura-base para discussão e aprovação. Ela não autoriza implementação automática. Após aprovação, a Fase 0 deverá transformar as decisões em ADRs, contratos e critérios verificáveis.



PARTE

# Developer Experience

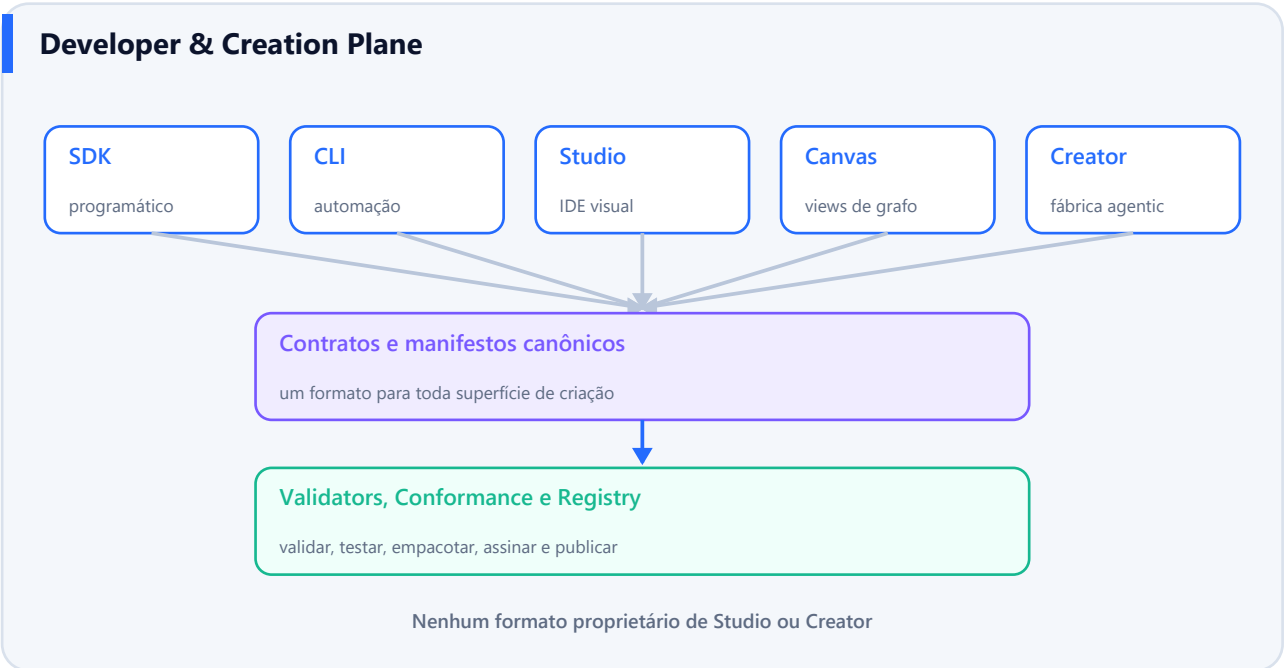
SDK, CLI, Studio, Canvas, marketplace, Creator e os modos operacionais.

# 06



DEVELOPER EXPERIENCE

## 23. Developer & Creation Plane



A arquitetura só se torna ecossistema quando extensões corretas podem ser criadas rapidamente. O Developer & Creation Plane é a camada que transforma contratos internos em jornadas de produto para desenvolvedores, operadores e criadores.

### 23.1 Invariante de formato único

SDK, CLI, Studio, Canvas e Creator devem ler e escrever os mesmos manifests, schemas, workflows, lockfiles, policies e evaluation profiles. Não haverá formato visual proprietário nem uma API secreta para o Creator.

REXO Studio + REXO CLI + REXO SDK + REXO Creator → Canonical Contracts → Validators → Registry

### 23.2 Componentes

| Componente   | Responsabilidade                                                   |
|--------------|--------------------------------------------------------------------|
| REXO SDK     | APIs, builders, clients, testing e conformance para extensões      |
| REXO CLI     | Scaffolding, validação, teste, packaging, instalação e publicação  |
| Generators   | Estruturas iniciais consistentes para cada tipo de componente      |
| Validators   | Schemas, semantics, permissions, compatibility e quality baselines |
| REXO Studio  | IDE visual, console operacional, debugger e governance workspace   |
| REXO Canvas  | Views gráficas de workflows, capabilities e entidades de domínio   |
| REXO Creator | Platform Factory agentic para gerar candidatos a ecossistemas      |

## 23.3 Regra de extensibilidade

Uma extensão só é válida quando pode ser criada por arquivo, SDK, CLI ou Studio e produzir uma representação canônica equivalente. Qualquer recurso disponível apenas em uma superfície será considerado dívida arquitetural.

### CONSTRUCTION KIT

## 24. REXO SDK

O SDK é o kit oficial de construção do ecossistema. Ele não será apenas um cliente HTTP: fornecerá contratos, builders, runtime clients, mocks, harness local, validators, test fixtures, conformance suites e ferramentas de packaging.

```
rexo-sdk/  
|-- contracts/  
|-- manifest-builders/  
|-- capability-api/  
|-- provider-api/  
|-- workflow-dsl/  
|-- agent-composition/  
|-- memory-api/  
|-- policy-api/  
|-- evaluation-api/  
|-- package-api/  
|-- distribution-api/  
|-- local-harness/  
|-- mocks/  
|-- validators/  
`-- conformance-kit/
```

### 24.1 APIs conceituais

| Builder              | Resultado                                                            |
|----------------------|----------------------------------------------------------------------|
| createCapability()   | Declara contrato abstrato, schemas, efeitos, risco e quality profile |
| createProvider()     | Registra implementação concreta e health/cost metadata               |
| createSkill()        | Empacota competência, recursos, testes e evals                       |
| createWorkflow()     | Compõe DAG durável, gates, retries e migrations                      |
| createPack()         | Agrupar componentes, dependências, permissions e lifecycle           |
| createApp()          | Cria experiência que consome capabilities                            |
| createDistribution() | Compõe packs, providers, policies e Apps                             |
| createEvaluator()    | Define avaliação executável e baseline                               |

### 24.2 Estratégia de linguagens

Schemas e manifests serão language-neutral. SDKs serão gerados a partir dos contratos canônicos. A escolha das primeiras linguagens será feita por ADR, mas a arquitetura assume ao menos uma experiência forte para automação e outra para aplicações.

## 24.3 Conformance Test Kit

- Valida schemas, ids, versões, dependencies e capability semantics.
- Executa exemplos e golden tests.
- Testa idempotência, retries, cancellation e timeout.
- Simula falta de provider, falha parcial e rollback.
- Verifica permissions manifest e ausência de acesso excedente.
- Produz relatório publicável no marketplace.

### DEVELOPER WORKFLOW

## 25. REXO CLI, scaffolding e generators

A CLI será a primeira superfície de desenvolvimento e automação. O Studio consumirá os mesmos serviços e contratos, evitando divergência entre experiência visual e código.

### 25.1 Interface alvo

Os comandos abaixo definem a experiência futura desejada. Eles ainda não existem nesta fase arquitetural.

```
rexo create capability media.generate-video
rexo create provider specialist-video-provider
rexo create workflow course-production
rexo create pack education-assessments
rexo create distribution legal-rexo

rexo validate    rexo test    rexo eval
rexo pack       rexo install education    rexo publish
```

### 25.2 Jornada de criação de Pack

1. Selecionar tipo de Pack e namespace.
2. Gerar scaffold com manifesto, schemas, exemplos, tests e evals.
3. Adicionar capabilities fornecidas e dependências.
4. Declarar permissions, data policy e runtime requirements.
5. Executar validação estática e semântica.
6. Executar testes locais em sandbox.
7. Executar conformance e regression suite.
8. Gerar pacote imutável, SBOM, assinatura e changelog.
9. Publicar em canal development ou canary.
10. Promover a stable após critérios de confiança.

### 25.3 Generators

Generators oferecem estruturas iniciais opinionated, mas não escondem contratos. Templates são versionados, substituíveis e testados. Atualizações de generator não modificam componentes existentes silenciosamente.

VISUAL PRODUCT

## 26. REXO Studio e REXO Canvas

REXO Studio será uma interface visual unificada inspirada em IDEs e engines criativas. REXO Canvas será o conjunto de views gráficas dentro do Studio, não um produto separado.

| Modo do Studio   | Capacidades                                                                    |
|------------------|--------------------------------------------------------------------------------|
| Design Mode      | Workflows, Packs, capabilities, providers, templates, policies e distributions |
| Run Mode         | Execução ao vivo, pause, resume, approvals, artifacts e custos                 |
| Debug Mode       | Task envelopes, context, resolution, retries, fallbacks e replay               |
| Governance Mode  | Permissions, packages, trust, audit, retention e release                       |
| Marketplace Mode | Discovery, comparison, install plans, updates e rollback                       |

### 26.1 Workflow Canvas

Usuários conectam capabilities visualmente. Cada aresta representa um contrato de artifact ou event. O editor impede conexões incompatíveis, exige gates para efeitos críticos e gera o mesmo Workflow Definition usado pela CLI.

Research → Curriculum → Slides → Video → QA → Publish

### 26.2 Domain Canvas

O Domain Canvas representa entidades e artifacts, não apenas tarefas. Para Education, o usuário manipula Curso, Módulo, Aula, Vídeo, PDF, Imagem, Quiz e Exercício.

- Reordenar, duplicar e criar dependências.
- Bloquear artifacts aprovados contra regeneração.
- Trocar provider apenas para um nó.
- Comparar versões e candidatos.
- Reexecutar somente uma subárvore.
- Visualizar proveniência, custo, qualidade e status.
- Publicar uma versão consistente do conjunto.

### 26.3 Round-trip fidelity

Abrir um workflow criado pela CLI no Studio e salvá-lo não pode alterar sua semântica. Alterações visuais devem produzir diffs legíveis e revisáveis.

DISTRIBUTION

## 27. Runtime Marketplace e lifecycle de instalação

O ecossistema terá três catálogos coordenados: Package Marketplace, Runtime Catalog e Distribution Marketplace.

| Catálogo                 | Distribui                                                                   |
|--------------------------|-----------------------------------------------------------------------------|
| Package Marketplace      | Packs, Skills, workflows, Apps, templates, evaluators, policies e providers |
| Runtime Catalog          | Worker profiles, sandboxes, ambientes, dependências nativas e perfis de GPU |
| Distribution Marketplace | Plataformas completas compostas e versionadas                               |

### 27.1 Instalação transacional

1. Resolver distribuição, packs, providers, runtimes e policies.
2. Verificar assinatura, publisher, trust tier, SBOM e vulnerabilidades.
3. Calcular permission diff, requisitos de hardware, licenças e custo.
4. Detectar conflitos e construir install plan.
5. Solicitar aprovação quando permissions ou efeitos exigirem.
6. Baixar e verificar artifacts por hash.
7. Instalar em staging e executar migrations reversíveis.
8. Executar smoke tests e conformance.
9. Ativar atomicamente e gravar lockfile.
10. Manter plano de rollback e versão anterior quando aplicável.

### 27.2 Atualização e remoção

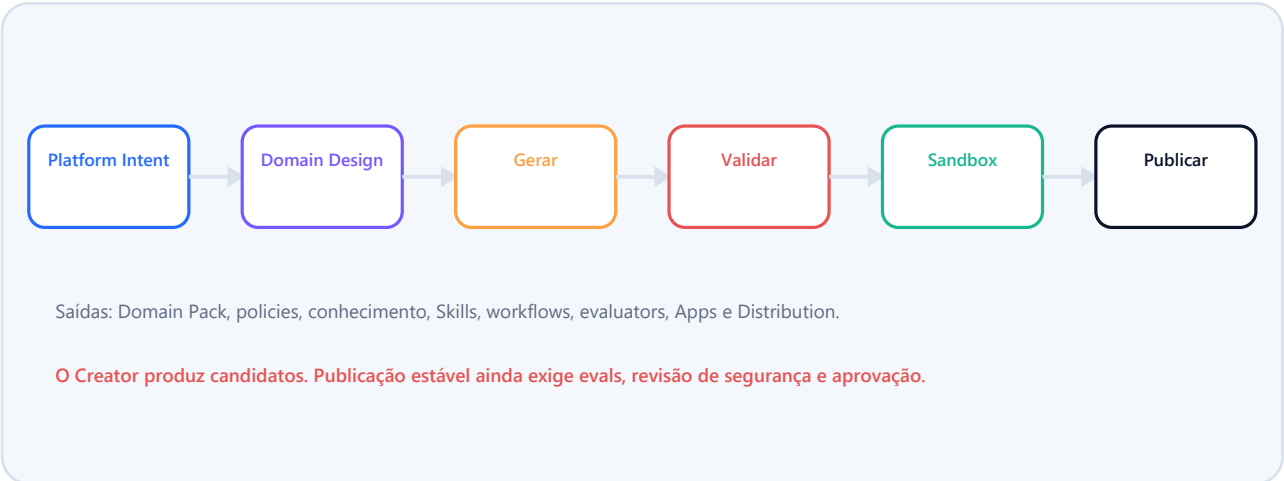
Updates produzem compatibility report, migration plan e permission diff. Remoção verifica dependências, preserva dados conforme policy e executa uninstall plan sem apagar artifacts compartilhados.

### 27.3 Runtime Profiles

Packs podem requerer capacidades de runtime, mas não devem executar installers arbitrários no host. Perfis de runtime serão assinados, reproduzíveis, isolados e selecionados pelo Scheduler.

#### PLATFORM FACTORY

## 28. REXO Creator



REXO Creator é uma aplicação agentic construída sobre o SDK. Ele recebe uma Platform Intent Specification e produz um candidato completo a distribuição especializada.

## 28.1 Platform Intent Specification

- Domínio, jurisdição, público e casos de uso.
- Riscos, dados, privacidade e compliance.
- Capabilities esperadas e resultados proibidos.
- Providers disponíveis e restrições de implantação.
- Qualidade, budgets, autonomia e gates humanos.
- Apps e experiências desejadas.

## 28.2 Outputs

- Domain Pack e taxonomia.
- Capability Packs e Provider requirements.
- Policies, threat model e data classification.
- Knowledge ingestion pipelines.
- Skills, workflows, templates e evaluators.
- Apps e interfaces iniciais.
- Distribution manifest, lockfile de desenvolvimento e documentação.
- Evals, golden datasets e plano de evolução.

## 28.3 Limites

Creator não modifica o Core, não reduz políticas mandatórias, não publica diretamente em stable e não concede a si próprio novas permissões. Seus outputs passam por validators, security review, evals, sandbox, canary e promoção.

## 28.4 Exemplo

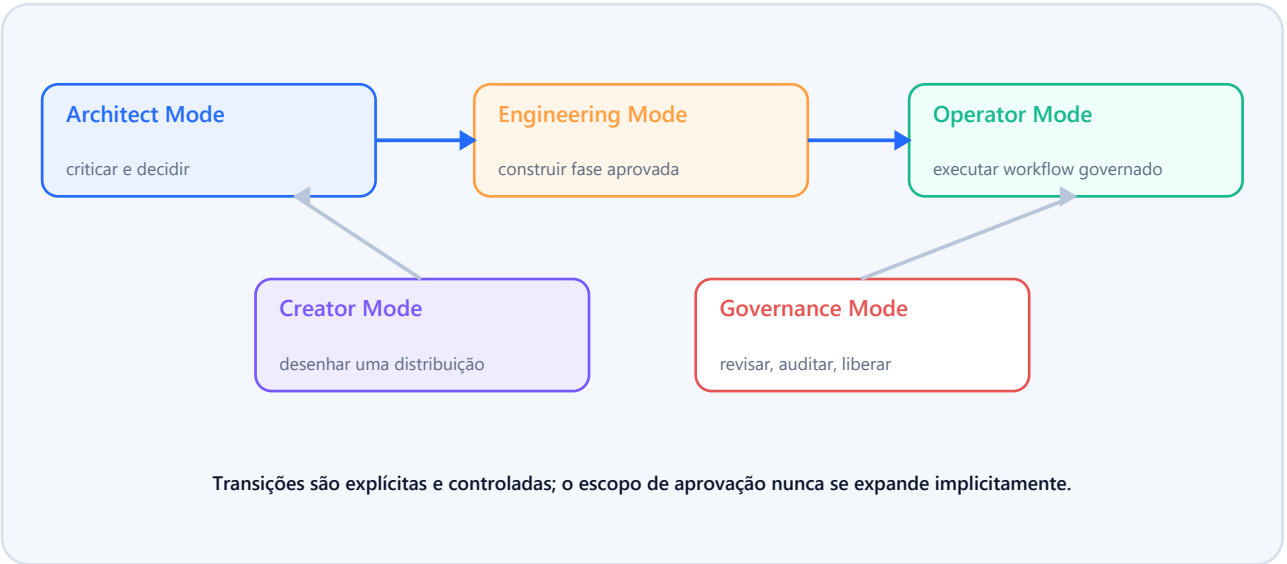
Entrada: sistema especializado em advocacia brasileira para pequenos escritórios. Saída: Legal Domain Pack, Brazilian Law Knowledge Pack, Legal Policy Pack, Contract Analysis Pack, research workflows, legal evaluators, workspace App e REXO Legal Distribution.

### COMO O REXO FUNCIONA

# 29. Modos operacionais e ativação

---





Modo operacional é um perfil explícito de autoridade, comportamento, outputs e stop conditions. Ele não é uma personalidade livre nem aumenta permissões automaticamente.

| Modo                | Comportamento                                                                  |
|---------------------|--------------------------------------------------------------------------------|
| Chief AI Architect  | Critica, compara alternativas, produz ADRs/RFCs e não implementa sem aprovação |
| Ecosystem Designer  | Define contratos, SDK, Packs, providers, marketplace e Developer Experience    |
| Product Engineering | Implementa apenas a fase e o escopo aprovados, com testes e evidência          |
| Operator            | Executa workflows publicados sob policies, budgets e gates                     |
| Creator             | Gera candidatos a novas plataformas e distributions                            |
| Governance          | Audita, avalia, aprova, promove, bloqueia ou reverte releases                  |

### 29.1 Como ativar comigo agora

Hoje, antes da existência do runtime REXO, o modo é um contrato de trabalho nesta conversa. Ele é ativado por instrução explícita em linguagem natural. Não existe um botão oculto ou comando instalado.

**Arquitetura:**  
Ative o modo Chief AI Architect para este projeto. Critique a proposta, produza decisões e não implemente arquivos.

**Engenharia:**  
Aprovo a arquitetura v2. Ative Product Engineering somente para a Fase 0, seguindo os gates do documento.

**Creator:**  
Ative REXO Creator para projetar uma distribuição jurídica. Gere apenas a especificação candidata e os riscos.

**Operação:**  
Ative Operator Mode para executar o workflow X com budget Y e aprovação antes de publicação.

### 29.2 Confirmação de ativação

Ao ativar um modo, o sistema deve responder com Mode Activation Record contendo: modo, objetivo, escopo, autoridade, restrições, artifacts esperados, budgets, gates, stop conditions e estado inicial.

## 29.3 Interface futura da CLI

Os comandos abaixo são parte da especificação alvo e não estão implementados atualmente.

```
rexo mode activate architect --project .
rexo mode activate engineering --phase 0
rexo mode activate creator --intent legal-platform.yaml
rexo mode status
rexo mode transition engineering --approval ADR-SET-001
rexo mode deactivate
```

## 29.4 Máquina de estados

DRAFT → ARCHITECTURE\_REVIEW → ARCHITECTURE\_APPROVED → PHASE\_PLANNED → BUILDING → VALIDATING  
→ CANARY → STABLE

- Transições exigem evidência e aprovação definidas pela policy.
- Aprovar arquitetura não autoriza todas as fases de implementação.
- Aprovar uma fase não autoriza publicação externa.
- Novo objetivo material cria nova ativação ou amendment.
- Modo pode ser pausado, revogado ou reduzido a qualquer momento.
- Falhas graves transitam para BLOCKED, ROLLBACK ou INCIDENT.

## 29.5 Como eu funcionarei no projeto

| Momento       | Obrigação                                                                     |
|---------------|-------------------------------------------------------------------------------|
| Antes de agir | Ler estado, modo, contratos, ADRs, policies, orçamento e artifacts relevantes |
| Ao planejar   | Produzir tarefas, dependências, riscos, critérios e gates verificáveis        |
| Ao decidir    | Explicar alternativas e registrar decisões arquiteturais materiais            |
| Ao executar   | Usar apenas ferramentas, dados e escopo autorizados                           |
| Ao revisar    | Aplicar evals independentes e corrigir falhas específicas                     |
| Ao aprender   | Registrar candidatos; não promover conhecimento automaticamente               |
| Ao concluir   | Entregar artifacts, evidências, limitações, custos e próximo estado           |



PARTE

# Apêndices

Glossário, checklist de aprovação e o próximo passo autorizado.

07



REFERÊNCIA

# Apêndice A. Glossário

| Termo            | Definição                                                            |
|------------------|----------------------------------------------------------------------|
| REXO Core        | Núcleo estável do runtime e dos serviços de plataforma.              |
| Capability       | Resultado abstrato que pode possuir múltiplas implementações.        |
| Implementation   | Forma concreta de satisfazer uma capability.                         |
| Provider         | Integração com modelo, serviço, MCP, API, CLI ou ferramenta local.   |
| Skill            | Unidade versionada de competência reutilizável.                      |
| Agent Definition | Papel lógico composto por Skills, policies e capacidades de decisão. |
| Worker           | Instância efêmera que executa uma task.                              |
| Workflow         | DAG durável de tasks, gates, avaliações e compensações.              |
| Pack             | Unidade instalável e versionada do ecossistema.                      |
| App              | Experiência orientada ao usuário construída sobre capabilities.      |
| Distribution     | Composição de Core, packs, providers, policies e apps.               |
| Artifact         | Output versionado e rastreável produzido pelo sistema.               |
| Run              | Execução de um objetivo completo.                                    |
| Task             | Unidade agendada de trabalho dentro de uma run.                      |
| Eval             | Avaliação executável de qualidade, segurança ou desempenho.          |
| Policy           | Regra aplicada pelo sistema independentemente do prompt.             |
| Provenance       | Histórico de origem, transformação e autoria de dados e artifacts.   |
| REXO SDK         | Kit oficial para construir, testar e empacotar extensões.            |
| REXO Studio      | Interface visual unificada de design, operação, debug e governança.  |
| REXO Canvas      | Conjunto de views gráficas dentro do Studio.                         |
| REXO Creator     | Platform Factory agentic que gera distribuições candidatas.          |
| Operating Mode   | Perfil explícito de autoridade, comportamento e stop conditions.     |

PRÓXIMO PASSO

# Apêndice B. Checklist de aprovação

- ☐ A definição de REXO representa corretamente o produto pretendido?
- ☐ Capability-first é aceito como princípio central?
- ☐ Agentes são aceitos como composições opcionais, não unidade fundamental?
- ☐ A separação Core, Packs, Apps e Distributions está aprovada?
- ☐ REXO Education está aprovado como primeiro vertical?

- □ Marketplace público permanece posterior à maturidade de segurança?
- □ Monólito modular é aceito como topologia inicial?
- □ Memória governada e promoção controlada estão aprovadas?
- □ Tool Gateway central e capability tokens estão aprovados?
- □ A sequência de fases está adequada às prioridades do projeto?
- □ Developer & Creation Plane é aceito como camada oficial?
- □ SDK, CLI, Studio e Creator devem produzir o mesmo formato canônico?
- □ REXO Canvas deve permanecer uma view dentro do Studio?
- □ Runtime Marketplace e instalação transacional estão aprovados?
- □ Os modos operacionais e seu protocolo de ativação estão aprovados?

ENCERRAMENTO

# Próximo passo

A aprovação desta Constituição não inicia automaticamente a implementação. Ela autoriza a Fase 0, cujo objetivo é converter a visão em contratos, decisões e critérios verificáveis.

| Entrega da Fase 0  | Resultado esperado                                                          |
|--------------------|-----------------------------------------------------------------------------|
| ADRs               | Decisões arquiteturais individualizadas, alternativas e consequências       |
| Contratos          | Schemas iniciais de Goal, Task, Capability, Artifact, Eval, Event e Package |
| Threat model       | Trust boundaries, riscos, controles e gates obrigatórios                    |
| Taxonomia          | Namespaces e convenções oficiais de capabilities                            |
| DX Specification   | SDK, CLI, scaffolding, Studio, Canvas, marketplace e Creator                |
| Roadmap executável | Epics, dependências, critérios de conclusão e ordem de construção           |

Arquitetura universal. Primeira implementação vertical. Evolução orientada por evidência.